

Marissa Ratschki

CMPT 308

Lab 05

PART A:

A1:

```
SELECT student_id, name
FROM Students
WHERE student_id IN (
    SELECT student_id
    FROM Enrollments
    WHERE course_id = 'CMPT308'
    AND term = '2026SP'
)
ORDER BY student_id;
```

Output:

25	"Artyom"
29	"Mila"
57	"John"

A2:

```
SELECT c.course_id, c.title
FROM Courses c
WHERE EXISTS (
    SELECT 1
    FROM Enrollments e
    WHERE e.course_id = c.course_id
    AND e.term = '2026SP'
)
ORDER BY c.course_id;
```

Output:

"CMPT308"	"Database Management"
"CYBR210"	"Network Security"

A3:

```
SELECT s.student_id, s.name
FROM Students s
WHERE NOT EXISTS (
  SELECT 1
  FROM Enrollments e
  WHERE e.student_id = s.student_id
  AND e.term = '2026SP'
)
ORDER BY s.student_id;
```

Output:

34	"Lucy"
55	"Tyler"

A4:

```
SELECT student_id
FROM Enrollments
WHERE course_id = 'CMPT308'
AND term = '2026SP'
```

UNION

```
SELECT student_id
FROM Enrollments
WHERE course_id = 'CYBR210'
AND term = '2026SP'
```

```
ORDER BY student_id;
```

Output:

25
29
48
57

A5:

```
SELECT student_id
FROM Enrollments
WHERE course_id = 'CMPT308'
AND term = '2026SP'
```

INTERSECT

```
SELECT student_id
FROM Enrollments
WHERE course_id = 'CYBR210'
AND term = '2026SP'
```

ORDER BY student_id;

Output:

57

A6:

```
SELECT student_id
FROM Enrollments
WHERE course_id = 'CMPT308'
AND term = '2026SP'
```

EXCEPT

```
SELECT student_id
FROM Enrollments
WHERE course_id = 'CYBR210'
AND term = '2026SP'
```

ORDER BY student_id;

Output:

25

29

PART B: Rules: A → B

B → C

AC → D

D → E

B1:

$A^+ = \{A\}$

Using $A \rightarrow B$, since A is already in closure, add B:

$A^+ = \{A, B\}$

Using $B \rightarrow C$, since B is now in closure, add C:

$A^+ = \{A, B, C\}$

Using $AC \rightarrow D$, since both A and C are now in closure, add D:

$A^+ = \{A, B, C, D\}$

Using $D \rightarrow E$, since D is now in closure, add E:

$A^+ = \{A, B, C, D, E\}$

B2:

$(A, C)^+ = \{A, C\}$

Using $A \rightarrow B$, since A is already in the closure, add B:

$(A, C)^+ = \{A, B, C\}$

Using $B \rightarrow C$ does not add anything new because C is already in the closure.

Using $AC \rightarrow D$, since both A and C are in the closure, add D:

$(A, C)^+ = \{A, B, C, D\}$

Using $D \rightarrow E$, since D is now in the closure, add E:

$(A, C)^+ = \{A, B, C, D, E\}$

B3:

A candidate key for R is A. This is because A determines all attributes in the relation. It also possesses the minimal number of attributes to ID the row. Together, this makes A a candidate key.

PART C:**C1: Is EnrollmentWide in 1NF?**

Yes, EnrollmentWide is in 1NF as each column has atomic values, representing one student enrolled in one course during one term, and each field stores a singular value.

C2: Does EnrollmentWide violate 2NF?

Yes, EnrollmentWide violates 2NF as it has partial dependencies. The primary key is composite: (student_id, course_id, term). However, for example, student_name depends only on student_id, not on the full composite key.

Partial dependencies:

- student_id → student_name
- course_id → course_title

C3: Give a 2NF decomposition

A 2NF decomposition is:

Students(student_id, student_name)

Primary Key: student_id

Courses(course_id, course_title)

Primary Key: course_id

Enrollments(student_id, course_id, term, grade)

Primary Key: (student_id, course_id, term)

PART D:

D1: Explain why this table has redundancy and can cause update anomalies

This table has redundancy because instructor_name may be repeated in multiple rows in the case that the instructor teaches more than one course. If an instructor's name changes, every row with that instructor would need to be updated. Or, if only some rows are updated in an instance, the table could contain outdated instructor names.

D2: Is it in 3NF?

This table is not in 3NF, as it contains a transitive dependency. The key is course_id because course_id determines course_title and instructor_id, and instructor_id determines instructor_name. This creates the transitive dependency course_id -> instructor_id -> instructor_name.

D3: Give a 3NF decomposition

Courses(course_id, course_title, instructor_id) – PK is course_id, FK is instructor_id

Instructors(instructor_id, instructor_name) – PK is instructor_id

D4: Extra credit: Is the decomposition lossless?

Yes, this decomposition is lossless. Using the intersection test idea, the shared attribute, being instructor_id, determines one side of the decomposition, so the original information can be reconstructed by joining the tables back together.